

[← Go Back](#)

Hardware

[Edge Impulse with the Nano 33 BLE Sense](#)[Gesture Recognition with the Nano 33 BLE Sense](#)[Get Started With Machine Learning on Arduino](#)[Getting Started with OpenMV](#)[Reading Temperature & Humidity on Nano 33 BLE Sense](#)[Connecting Two Nano 33 BLE Sense Boards Through I2C](#)[Accessing Accelerometer Data on Nano 33 BLE Sense](#)[Accessing Gyroscope Data on Nano 33 BLE Sense](#)[Accessing Magnetometer Data on Nano 33 BLE Sense](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)[Controlling the On-Board RGB LED with Microphone](#)[Proximity Detection with the Nano 33 BLE Sense](#)[Detecting Colors with the Nano 33 BLE Sense Board](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Connecting Two Nano 33 BLE Sense Boards Through UART](#)

Home / Hardware / Nano 33 BLE Sense / **Controlling the On-Board RGB LED with Microphone**



This tutorial refers to a product that has reached its end-of-life status.

Controlling the On-Board RGB LED with Microphone

Learn how to create a soundmeter using the built-in microphone on the Nano 33 BLE Sense.

Author · Fabricio Troya · Last revision · 07/17/2024

In this tutorial we will use an **Arduino Nano 33 BLE Sense** board to measure and display the sound values of your surroundings, made possible by the embedded **MP34DT05** sensor.

Warning: A very small percentage of people may experience a seizure when exposed to flashing lights or patterns. Even people who have no history of seizures or epilepsy may have an undiagnosed condition that can cause these “photosensitive epileptic seizures” while watching lights blinking fast. Immediately stop and consult a doctor if you experience any symptoms.

ON THIS PAGE

Goals

[Hardware & Software Needed](#)[The MP34DT05 Sensor](#)[Creating the Program](#)[Testing It Out](#)[+](#)[Conclusion](#)[Help](#)

Goals

The goals of this project are:

- Learn how to output raw sensor data from the Arduino Nano 33 BLE Sense.

- Use the PDM(Pulse-density modulation) library.

- Print sound values in the Serial Monitor.

- Create your own RGB sound meter.

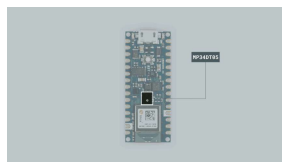
Hardware & Software Needed

Arduino Nano 33 BLE Sense.

This project uses no external sensors or components.

In this tutorial we will use the [Arduino Cloud Editor](#) to program the board.

The MP34DT05 Sensor



The MP34DT05 microphone sensor.

Microphones are components that convert physical sound into digital data. Microphones are commonly used in mobile terminals, speech recognition

The MP34DT05 sensor is a ultra-compact microphone that use PDM (Pulse-Density Modulation) to represent an analog signal with a binary signal. The sensor's range of different values are the following:

Signal-to-noise ratio: 64dB

Sensitivity: -26dBFS $\pm$ 3dB

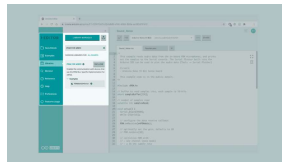
Temperature range: -40 to 85°C

If you want to read more about the MP34DT05 sensor you can take a look at the [datasheet](#).

Creating the Program

1. Setting up

Let's start by opening the [Arduino Cloud Editor](#), click on the **Libraries** tab, search for the **PDM FOR MBED** library, then in **Examples**, open the **PDMSerialPlotter** example. Once the sketch is open, rename it as **Sound_Meter**.

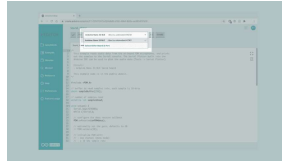


Finding the library in the Cloud Editor.

2. Connecting the board

Now, connect the Arduino Nano 33 BLE Sense to the computer to check that the Cloud Editor

the image. If they don't appear, follow the [instructions](#) to install the plugin that will allow the Editor to recognize your board.



Selecting the board.

3. Printing and displaying sound values

Now we will need to modify the code of the example, in order to print the sound values and turn on a different LED based how noisy is the sound.

Before the `setup()` there are two types of variables initialized. One is a `short` variable and the other is a `volatile int` variable. We use the `short` type variable to store 16-bit data-types as the `sampleBuffer[256]`. The other, `volatile`, is a keyword known as a variable qualifier. It is usually used before the datatype of a variable, in order to modify the way in which the compiler and subsequent program treat the variable. In this case, it directs the compiler to load the variable `samplesRead` from RAM and not from a storage register.

```
1 #include <PDM.h>
2
3 // buffer to read sample
4 short sampleBuffer[256]
5
6 // number of samples read
7 volatile int samplesRead
```

In the `setup()`, we use the `PDM.onReceive()` function to configure the data receive callback. Lastly, the `PDM.begin()` sets the sensor to read data from just one channel and a sample rate of 16 kHz, this statement is inside an `if()` that will print a message, as a string, in case the sensor has not been properly initialized.

```
1 void setup() {
2   Serial.begin(9600);
3   while (!Serial);
4
5   // configure the data receive
6   PDM.onReceive(onPDMDataReceived);
7
8   // optionally set the gain
9   // PDM.setGain(30);
10
11  // initialize PDM with
12  // - one channel (mono)
13  // - a 16 kHz sample rate
14  if (!PDM.begin(1, 16000)) {
15    Serial.println("Failed to initialize PDM");
16    while (1);
17  }
18 }
```

Then, in the `loop()`, let's modify the example code by adding the following portion of code inside the `for()` loop, after the

```
1 // check if the sound
2   if (sampleBuffer[
3       digitalWrite(LED
4       digitalWrite(LED
5       digitalWrite(LED
6   }
7   // check if the s
8   if (sampleBuffer[
9       digitalWrite(LED
10      digitalWrite(LED
11      digitalWrite(LED
12  }
13  //check if the so
14  if (sampleBuffer[
15      digitalWrite(LED
16      digitalWrite(LED
17      digitalWrite(LED
18  }
```

With this portion of the code, we will turn on the RGB LED based on the amount of sound that the microphones is receiving.

4. Complete code

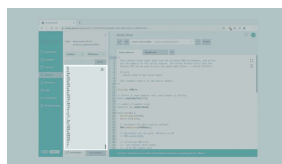
If you choose to skip the code building section, the complete code can be found below:



```
1  /*
2   This example reads
3   out the samples to
4   Arduino IDE can be
5
6   Circuit:
7   - Arduino Nano 33 E
8
9   This example code i
10 */
11
12 #include <PDM.h>
13
14 // buffer to read sam
15 short sampleBuffer[25
16
17 // number of samples
18 volatile int samplesF
19
20 void setup() {
21   Serial.begin(9600);
22   while (!Serial);
23
24   // configure the da
25   PDM.onReceive(onPDM
26
27   // optionally set t
28   // PDM.setGain(30);
29 }
```

Testing It Out

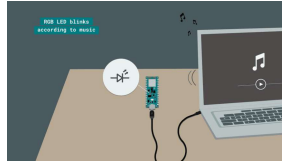
After you have successfully verified and uploaded the sketch to the board, open the Serial Monitor from the menu on the left. You will now see the new values printed.



Microphone data in the Serial Monitor.

If you want to test it, the only

the board next to a speaker and play some music to see how the colors of the RGB LED change based on the music.



RGB LED blinking according to the music.

Warning: Remember that depending of the music, lights might blink too fast. **Immediately stop playing and consult a doctor if you experience any symptoms of “photosensitive epileptic seizures”.**

Troubleshoot

Sometimes errors occur, if the project is not working as intended there are some common issues we can troubleshoot:

Missing a bracket or a semicolon.

If your Arduino board is not recognized, check that the Create plugin is running properly on your computer.

Accidental interruption of cable connection.

The sound of the music is too low or the board is too far from the speaker.

Conclusion

In this simple tutorial we learned how to read sound values from the **MP34DT05** connecting the

sensor embedded in the Arduino Nano 33 BLE Sense board in order to print out sound values from the environment and visualize them through the RGB LED.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

